Siena Evers

March 5, 2025

CMPE 310 Project 1 Report

How I came up with this code

I started this project with pseudo code. I put comments in my code that broke down what I needed to accomplish in each section. The basic pseudo code included the variables needed, user input, converting the ascii value in binary, implementing a loop, using xor to compare the values, storing the hamming distance to a variable, and lastly printing the results. I also left many comments as I was working on my code because this my first time using assembly for a project. I got my code to run without bugs but it does not fully work as intended.

Errors and How to Fix Them

While my code is functional and has no bugs it does not fully complete the project as intended. I understand what needs to be fixed in my program and why my current code does not work. My code has two problems I was not able to fully figure out.

The first problem is printing the hamming distance. Printing the hamming distance works fine if the distance is between 0-9. If the hamming distance is more than one digit it will print an ASCII character of that value. The reason this is the case is because before setting the variable hamDistance to edx, I added '0', which is character 48 of the ascii table. If edx was a number between 0-9 the number stored in hamDistance would be the decimal value 48-57, which are the ascii characters for numbers. I tried to create a loop or use printf from the C library, but I ran out of time to implement it.

```
152 stopLoop:
153     add edx, '0'                ;makes it so hamDistance can be printable
154     mov [hamDistance], edx      ;set hamDistance to edx
155     jmp exit
```

This is a screenshot of the code I was referring to.

The second problem was more complicated than the first one. The code will only function properly if the strings were imputed as binary 1s and 0s. This problem had to do with the loop and method of comparing with the xor.

All characters are stored as eight binary digits. When saving each byte to al and bl there are eight binary digits that need to be compared. In my loop, I used an xor once and just ran one comparison when I should be comparing each bit. I do not know how to isolate the bits. I believe I would have to use shifting but my loop structure would have to be altered so I did not end up figuring out. The reason this code works if you type the binary of the strings is because the binary for the byte of 0 is 0000 0000 and 1 is 0000 0001. This means that the xor value can only be 1 or 0 because the 7 bits are all zero except for potentially the least significant bit. When typing in the string as binary you already type all the characters so the loop runs the right amount of times.

```
129     mov al, byte [foo + ecx] ;ecx increments through each byte of the string
130     mov bl, byte [bar + ecx]
```

This is each byte being stored int al, and bl.

```
140     xor al, bl          ;al will be set to 1 if al and bl don't match
141     cmp al, 1           ;if this is 1 then it should increment
142     je increment
```

using xor on both bytes in most circumstances ends up not as a plain 1.

example (this is not representing real code):

al = 0110 1111

bl = 0110 0001

xor al, bl ; would set al to = 0000 1110 ; which is not one

;this would require the increment function to loop 3 times, but

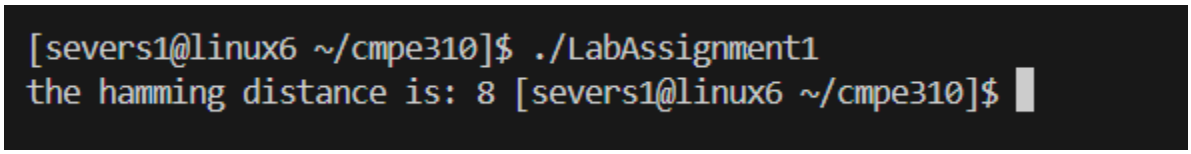
cmp al, 1 ;it's only checking if it equals 1, so it would not increment at all

Output of the Code

*In the code I left comments that have the binary values of the strings as well.

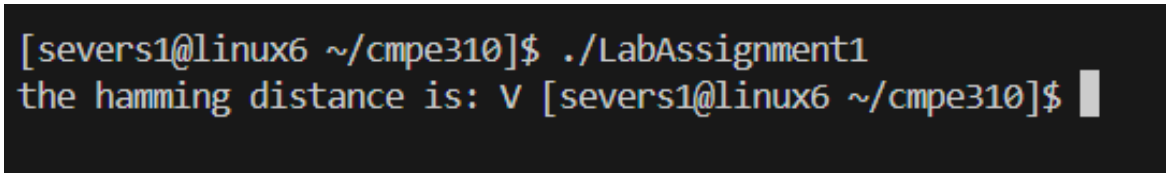
Output for foo and bar

- "foo" in binary "0110 0110 0110 1111 0110 1111"
- "bar" in binary "0110 0010 0110 0001 0111 0010"



Output for "this is a test" and "of an emergency broadcast"

- "this is a test" in binary "01110100 01101000 01101001 01110011 00100000 01101001 01110011 00100000 01100001 00100000 01110100 01100101 01110011 01110100"
- "of the emergency broadcast" in binary "01101111 01100110 00100000 01110100 01101000 01100101 00100000 01100101 01101101 01100101 01110010 01100111 01100101 01101110 01100011 01111001 00100000 01100010 01110010 01101111 01100001 01100100 01100011 01100001 01110011 01110100"



The hamming distance is supposed to be 38. V has a value of 86 in ASCII and 0 has a value of 48 in ASCII. So 86-48=38 therefore the distance was calculated to be 38, which means it is correct.

Output for custom result

- string1 “000111000111”
- string2 “010000101110”

```
[severs1@linux6 ~/cmpe310]$ ./LabAssignment1
the hamming distance is: 7 [severs1@linux6 ~/cmpe310]$
```

CODE:

```
;CMPE 310 Project 1
;Siena Evers, Spring 2025


;       IMPORTANT !!!
; My code runs without errors but it does not function perfectly
;there are two constraints:
;   it only works with 0s and 1s as characters in the string
;   it does not print the distance properly if it is more than one digit

;for the second test:
;   print statement is ascii, a V means 38 ('0' is 48, 48+38= 86 which is
'V')

section .data ;define constant variables
    msg db 'the hamming distance is: ' ;
    len equ $-msg

    msg1 db 'Input a string: ', 0xA ;
    msgLen1 equ $-msg1

    newLine db ' ' 0xA ; new line
    newLineLen equ $-newLine

    errorMsg1 db 'too many characters in the string (limit is 255)', 0xA
;error msg
    lenE1 equ $-errorMsg1


    ;testing variables
    foo db "000111000111", 0   ;
    lenfoo equ $-foo ;copies the length of the first string

    bar db "010000101110", 0   ;
    lenbar equ $-bar ;copies the length of the string
```

```
; This code only works if the inputed strings are already represented as
1s and 0s
; below is the binary values of the strings listed on the assignment page

; "foo" in binary "0110 0110 0110 1111 0110 1111"
; "bar" in binary "0110 0010 0110 0001 0111 0010"

; "this is a test"             in binary "01110100 01101000 01101001
01110011 00100000 01101001 01110011 00100000 01100001 00100000 01110100
01100101 01110011 01110100"
; "of the emergency broadcast" in binary "01101111 01100110 00100000
01110100 01101000 01100101 00100000 01100101 01101101 01100101 01110010
01100111 01100101 01101110 01100011 01111001 00100000 01100010 01110010
01101111 01100001 01100100 01100011 01100001 01110011 01110100"




section .bss ;reserving space in memory for future data

    hamDistance resb 1  ;reserve space for the hamming distance int
    hamDisLen equ $-hamDistance

    word1 resb 64  ; user input
    word1LEN equ $-word1

    word2 resb 64     ; user input
    word2LEN equ $-word2



section .text

    global _start


_start:

; User input is commented out due to it not being needed
```

```
; ASK THE USER FOR TWO INPUT STRINGS
; STORE THE STRINGS INTO TWO VARIABLES

;    mov edx, msgLen1
;    mov ecx, msg1
;    mov ebx, 1
;    mov eax, 4
;    int 0x80

;    mov  eax, 3 ; sys_read
;    mov  ebx, 0 ; stdin
;    mov  ecx, word1 ; user input
;    mov  edx, word1LEN ; max length
;    int  0x80

;    mov edx, msgLen1
;    mov ecx, msg1
;    mov ebx, 1
;    mov eax, 4
;    int 0x80

;    mov  eax, 3 ; sys_read
;    mov  ebx, 0 ; stdin
;    mov  ecx, word2 ; user input
;    mov  edx, word2LEN ; max length
;    int  0x80


; CHECK LENGTH OF THE STRINGS (WILL BE USING THE SHORTER LENGTH STRING
LENGTH)
; BOTH LENGTHS SHOULD BE UNDER 255 CHARACTERS

;mov edx, word1LEN
;cmp edx, 255
;jg overCharLimit

;mov edx, word2LEN
;cmp edx, 255
;jg overCharLimit
```

```
; TURN THE STRINGS INTO THEIR ASCII VALUES

    ;one byte is 8 bits and 1 ASCII char (2 digit  hexadecimal)
    ;this means each letter will have 8 bits to compare

    ;could compare the msb and shift the bits over to compare all 8 per


; I never got this ^ to work




; FIND THE HAMMING DISTANCE BETWEEN THE TWO VALUES
; use XOR to find the differing bits
; use a for loop to count the number of '1' bits on the XOR'd value


    mov ecx, 0  ;i of the loop
    mov edx, 0  ;counter that goes up
    jmp loop

loop:
    mov al, byte [foo + ecx] ;ecx increments through each byte of the
string
    mov bl, byte [bar + ecx]


    cmp al, 0
    je stopLoop
    cmp bl, 0
    je stopLoop

    inc ecx

    xor al, bl          ;al will be set to 1 if al and bl don't match
    cmp al, 1           ;if this is 1 then it should increment
```

```
    je increment

    jmp loop


increment:
    inc edx ;increments edx
    jmp loop


stopLoop:
    add edx, '0'                ;makes it so hamDistance can be printable
    mov [hamDistance], edx      ;set hamDistance to edx
    jmp exit



; PRINT STATEMENTS (In exit)

exit:

;    mov edx, word1LEN
;    mov ecx, word1
;    mov ebx, 1
;    mov eax, 4
;    int 0x80


;    mov edx, word2LEN
;    mov ecx, word2
;    mov ebx, 1
;    mov eax, 4
;    int 0x80


    mov edx, len
    mov ecx, msg
    mov ebx, 1
    mov eax, 4
    int 0x80
```

```
    mov edx, hamDisLen      ;this function prints the ascii value, which
means if the number is over 9 it prints an character
    mov ecx, hamDistance
    mov ebx, 1
    mov eax, 4
    int 0x80

    mov edx, newLineLen
    mov ecx, newLine
    mov ebx, 1
    mov eax, 4
    int 0x80


    mov eax,1             ;system call number (sys_exit)
    int 0x80




overCharLimit:
    mov edx, lenE1
    mov ecx, errorMsg1
    mov ebx, 1
    mov eax, 4
    int 0x80

    mov eax,1             ;system call number (sys_exit)
    int 0x80
```